EXPERIMENT-74(HDOC)

C Programming — Enumerations

Program 1: Enumeration with Explicit Values Starting from 10

Step 1: Understand the Problem Statement

- Input: None — the program does not read anything.
- Output: Every enumerator with its assigned value: `A=10, B=11, C=12`.
- In an enumeration the first enumerator normally gets 0 and each following one gets the previous value plus 1. A different value can be chosen for any enumerator by writing `= value` after its name, so `enum Letter { A = 10, B = 11, C = 12 }` assigns the values explicitly, starting from 10.
- An enumerator is an integer constant, so it can be printed with `%d` and used as a loop counter: the loop runs `letter` from `A` (10) to `C` (12).
- The names are not stored by the enum, so the expression `'A' + (letter - A)` turns the values 10, 11 and 12 into the characters A, B and C for printing.

Rule(s) used: enum Letter { A = 10, B = 11, C = 12 } gives A = 10, B = 11 and C = 12. Because the values are consecutive, the character of an enumerator is 'A' + (value − 10), and the loop `for (letter = A; letter <= C; letter++)` visits 10, 11 and 12.

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	`No input`	A=10, B=11, C=12	A=10, B=11, C=12	Pass

Step 3: Algorithm

1. Start the program.
2. Declare `enum Letter` with the explicit values A = 10, B = 11 and C = 12.
3. Call `printLetters()`.
4. In `printLetters()`, set `letter = A` and repeat while `letter <= C`.
5. For every value except the first, print a comma and a space; then print the character `'A' + (letter - A)`, an equals sign and the value of `letter`.
6. Increase `letter` by 1 after each pass.
7. After the loop print a newline.
8. Stop the program.

Evaluation against Step 2 test case: the loop starts at A (10), prints A=10, moves to B (11) and prints , B=11, then moves to C (12) and prints , C=12; the next value 13 is greater than C, so the loop ends. The output is A=10, B=11, C=12 — identical to the Expected Output(s) column, confirming the algorithm is correct before coding.

Step 4: C Program

File: hdoc75_1.c

```c
/* HDOC Day-75, Program 1: Enumeration with explicit values starting from 10.

   Normally the first enumerator gets 0 and each next one gets the previous
   value + 1. A different value can be given to any enumerator by writing
   "= value". Here the values are assigned explicitly, starting from 10:
       A = 10, B = 11, C = 12
   The program takes no input and prints every enumerator with its value.
*/

#include <stdio.h>

/* Enumeration with explicit values */
enum Letter { A = 10, B = 11, C = 12 };

/* Function prototype */
void printLetters(void);

int main(void) {
    printLetters();
    return 0;
}

/* Prints each enumerator with its assigned value */
void printLetters(void) {
    enum Letter letter;

    for (letter = A; letter <= C; letter++) {
        if (letter != A)
            printf(", ");
        /* 'A' + (letter - A) turns the values 10, 11, 12 into A, B, C */
        printf("%c=%d", 'A' + (letter - A), letter);
    }
    printf("\n");
}
```

Step 5: Compile, Execute and Evaluate

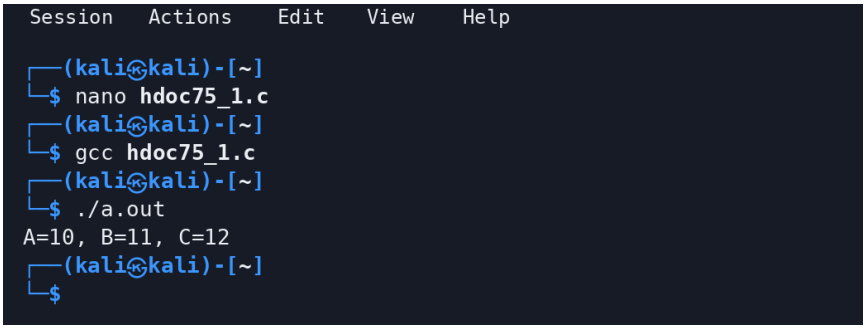
The program was compiled using the GCC compiler and executed from the Linux terminal using the following commands:

cc hdoc75_1.c	(compile)
./a.out	(execute)

All 1 test cases prepared in Step 2 were executed against the compiled program. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 6: Compilation and Execution Screenshot

Screenshot 1 of 1



Program 2: Menu Choices as an Enumeration (ADD, SUBTRACT, MULTIPLY) with switch

Step 1: Understand the Problem Statement

- Input: The operation name — ADD, SUBTRACT or MULTIPLY (any letter case) — followed by two integers, for example `ADD 10 20`.
- Output: The result of the operation, for example `30` for `ADD 10 20`. An unknown operation or non-integer numbers are rejected with an error message.
- The enumeration `enum Operation { ADD, SUBTRACT, MULTIPLY }` represents the menu choices and gives ADD = 0, SUBTRACT = 1 and MULTIPLY = 2.
- `parseOperation()` converts the typed name to the matching enumeration value: the name is changed to upper case with `toupper()` and compared with the three names using `strcmp()`. It returns 1 on success and 0 for an invalid name.
- `calculate()` receives the enumeration value and the two numbers and uses a `switch` statement: `case ADD` returns a + b, `case SUBTRACT` returns a − b and `case MULTIPLY` returns a × b. The numbers are stored as `long long` so that large results do not overflow.

Rule(s) used: ADD (0): result = a + b; SUBTRACT (1): result = a − b; MULTIPLY (2): result = a × b. The name is converted to upper case before comparing, so "add", "Add" and "ADD" all give the value 0.

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	`Input: ADD 10 20`	30	30	Pass
2	`Input: SUBTRACT 50 15`	35	35	Pass
3	`Input: MULTIPLY 6 7`	42	42	Pass
4	`Input: SUBTRACT 5 12`	-7	-7	Pass
5	`Input: multiply 3 -4`	-12	-12	Pass
6	`Input: DIVIDE 8 2`	Invalid operation. Enter ADD, SUBTRACT or MULTIPLY.	Invalid operation. Enter ADD, SUBTRACT or MULTIPLY.	Pass
7	`Input: ADD 5 x`	Invalid numbers. Enter two integers after the operation.	Invalid numbers. Enter two integers after the operation.	Pass

Step 3: Algorithm

1. Start the program.
2. Declare `enum Operation` with ADD, SUBTRACT and MULTIPLY (values 0, 1 and 2).

3. Read the operation name and call `parseOperation(name, &op)`: copy the name in upper case and compare it with "ADD", "SUBTRACT" and "MULTIPLY" using `strcmp()`; store the matching constant in op and return 1, or return 0 if nothing matches.

4. If `parseOperation()` returns 0, display `Invalid operation. Enter ADD, SUBTRACT or MULTIPLY.` and stop.

5. Read the two integers a and b; if they cannot be read, display `Invalid numbers. Enter two integers after the operation.` and stop.

6. Call `calculate(op, a, b)`: a `switch` on op returns `a + b` for ADD, `a - b` for SUBTRACT and `a * b` for MULTIPLY.

7. Display the returned result.

8. Stop the program.

Evaluation against Step 2 test cases: the input ADD 10 20 is converted to the enumeration value 0, so the switch returns 10 + 20 = 30. SUBTRACT 50 15 gives 35, MULTIPLY 6 7 gives 42 and SUBTRACT 5 12 gives −7 (a negative result is allowed). The lower-case input multiply 3 -4 is converted to MULTIPLY and gives −12. DIVIDE matches none of the three names and is rejected as an invalid operation, and ADD 5 x is rejected because x is not an integer — identical to the Expected Output(s) column, confirming the algorithm is correct before coding.

Step 4: C Program

File: hdoc75_2.c

```c
/* HDOC Day-75, Program 2: Enumeration for menu choices with a switch.

   enum Operation { ADD, SUBTRACT, MULTIPLY } represents the menu choices
   (ADD = 0, SUBTRACT = 1, MULTIPLY = 2). The user types the operation name
   followed by two integers, for example:  ADD 10 20
   The name is converted to the enumeration value and a switch statement on
   that value performs the operation.
*/

#include <stdio.h>
#include <string.h>
#include <ctype.h>

/* Enumeration for the menu choices */
enum Operation { ADD, SUBTRACT, MULTIPLY };

/* Function prototypes */
int       parseOperation(const char *text, enum Operation *op);
long long calculate(enum Operation op, long long a, long long b);

int main(void) {
    char name[20];
    long long a, b;
    enum Operation op;

    printf("Enter operation and two numbers (e.g. ADD 10 20): ");
    if (scanf("%19s", name) != 1 || !parseOperation(name, &op)) {
        printf("Invalid operation. Enter ADD, SUBTRACT or MULTIPLY.\n");
        return 1;
    }
    if (scanf("%lld %lld", &a, &b) != 2) {
        printf("Invalid numbers. Enter two integers after the operation.\n");
        return 1;
    }
```

```
    printf("%lld\n", calculate(op, a, b));
    return 0;
}

/* Converts the typed name (any letter case) to the enumeration value.
   Returns 1 on success and 0 if the name is not a valid operation. */
int parseOperation(const char *text, enum Operation *op) {
    char upper[20];
    int i;

    for (i = 0; text[i] != '\0' && i < 19; i++)
        upper[i] = (char)toupper((unsigned char)text[i]);
    upper[i] = '\0';

    if (strcmp(upper, "ADD") == 0)
        *op = ADD;
    else if (strcmp(upper, "SUBTRACT") == 0)
        *op = SUBTRACT;
    else if (strcmp(upper, "MULTIPLY") == 0)
        *op = MULTIPLY;
    else
        return 0;
    return 1;
}

/* Performs the operation selected by the enumeration value */
long long calculate(enum Operation op, long long a, long long b) {
    switch (op) {
        case ADD:
            return a + b;
        case SUBTRACT:
            return a - b;
        case MULTIPLY:
            return a * b;
    }
    return 0;
}
```

Step 5: Compile, Execute and Evaluate

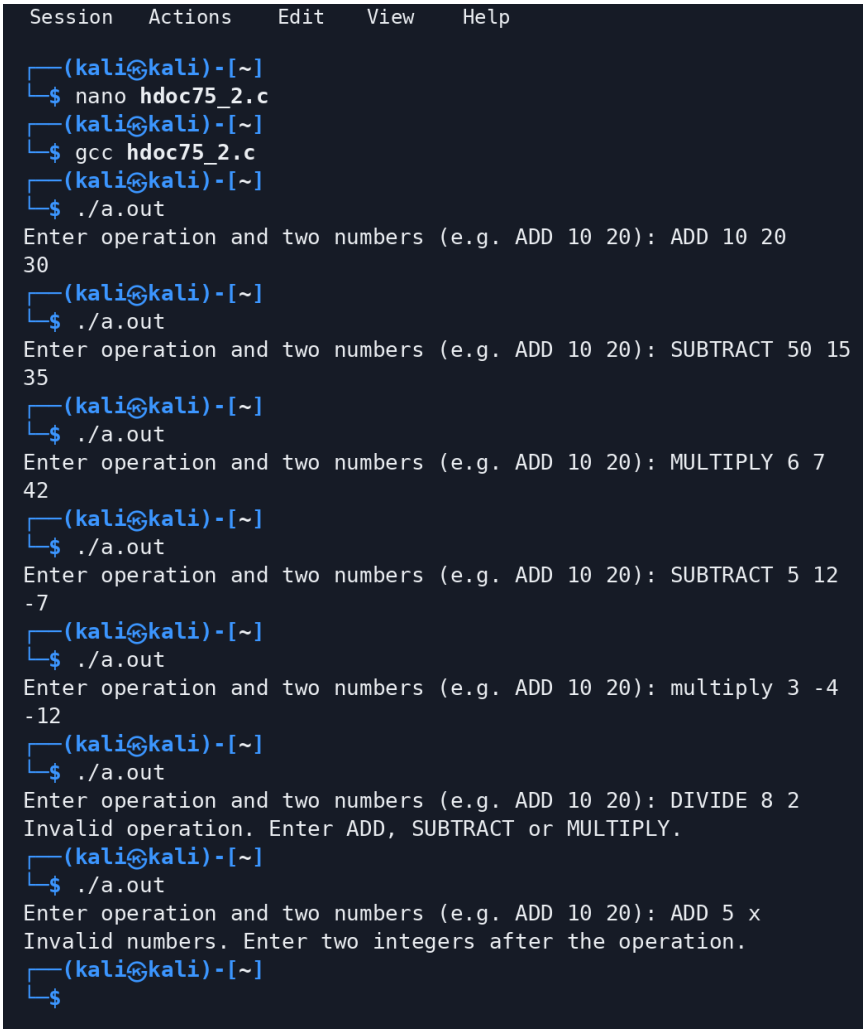
The program was compiled using the GCC compiler and executed from the Linux terminal using the following commands:

cc hdoc75_2.c	(compile)
./a.out	(execute)

All 7 test cases prepared in Step 2 were executed against the compiled program. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 6: Compilation and Execution Screenshot

Screenshot 1 of 1



Conclusion

In this experiment, enumerations were used to give meaningful names to integer constants. In the first program an enum was declared with explicit values starting from 10 (A = 10, B = 11, C = 12) and a loop printed each enumerator with its value. In the second program an enum for the menu choices ADD, SUBTRACT and MULTIPLY was used: the typed operation name was converted to its enum value and a switch statement performed the calculation on the two numbers entered (for example ADD 10 20 gave 30). Both programs were compiled with gcc and tested, including a lower-case input, negative results and invalid input, and every output matched the expected result.